

A self-labeling gate for premature agent stops

Juan Lescano

August 2026

Abstract

A coding agent with standing permission still stops early: it names the next step instead of taking it, offers a menu instead of picking, or asks the user to run what it could run itself. We describe a Stop-hook gate that catches those stops with three lanes of increasing cost, and a mechanism by which the gate labels its own errors from the transcript with no human in the loop. Over 150 real transcripts the gate went from 137 blocks to 127 once two false-positive classes were removed, with every freed case verified and no pending item lost. We report the negative results in full, including two failed attempts to obtain a usable confidence signal from the local model.

A coding agent with standing permission still stops early. It names the next step instead of taking it, offers a menu instead of picking, or asks the user to run what it could run itself. The user then spends their turn saying “continue”, which is the cheapest sentence in the transcript and the one that should never have to be written.

This is a report on a gate that catches those stops, and on the part that matters more: how the gate learns which of its own catches were wrong, without anyone reading a transcript.

1 What the gate is

A Stop hook runs when the agent finishes a turn. Exit 0 lets the turn end; exit 2 wakes the agent with the hook’s message. The gate is 8 checkers and a local model behind one orchestrator, 5389 lines of Python and 18 test files.

Three lanes decide, cheapest first.

Lane 0 - the audit. A message starting with `BLOCKED:` skips everything else. That made `BLOCKED:` the cheapest escape in the system, so it is now audited: a blocker declared with no tool call this turn, a list of four blockers at once, or three blocked closings in a row all fail the audit. A genuine blocker passes clean. An agent that survives the audit and still needs out gets a release phrase drawn from a CSPRNG, new per block, in no file it can read.

Lane 1 - the patterns. 298 regexes over the agent’s own words, grouped in classes: `ask`, `announce`, `wait`, `idle`. Quoted text does not count, so fenced blocks, markdown tables, backticks and reported speech are stripped before matching. An agent describing the gate is not using it.

Lane 2 - the local model. A 9B model over Ollama, one word out, `STOP` or `OK`, 0.7 to 0.9 seconds per verdict. It reads what the pattern cannot: whether “falta” means “I have work left” or “it was missing and I fixed it”.

2 Routing by confidence

Every pattern carries a confidence. A class written `ask:` condemns on its own; a class written `ask?:` only raises a doubt and hands the message to the model. 275 patterns are firm, 23 are doubts.

This split is what makes the model affordable. Most closings never reach it. The ones that do are exactly the ones a regex was going to get wrong.

3 Measurement

The gate was tuned against roughly 888 real closings pulled from live sessions, and A/B tested by running the old and new versions over the same 150 transcripts in parallel git worktrees.

Two false-positive classes dominated and were fixed:

class	share of what the model was condemning
harness error pages (API Error , usage limits)	11%
turns waiting on work the agent had already launched	19%

The second one is worth a note. The first attempt told the model about the background work in its prompt. Prose failed and few-shot examples failed: a 9B cannot hold an exception against its own “when unsure, answer STOP”. The fix was to stop asking. Waiting is now decided deterministically, before the model is called, by looking for a `<task-notification>` in the transcript.

Result over the 150 transcripts, same working directory on both sides: **137 blocks before, 127 after**. Every one of the 10 freed cases was verified by hand, and no real pending item was lost.

4 The part that closes the loop

The log says what the gate decided. It never said whether the decision was right, so tuning meant reading transcripts by hand.

The label was already in the transcript. A block wakes the agent, and the agent either goes and works or writes another paragraph and stops again. **Tool calls between the block and the next time the user speaks are the label**. Nobody has to supply it.

The report joins each logged decision back to its transcript, counts those tool calls, and groups by pattern class and by model confidence. A pattern that reaches 8 blocks with 70% of them buying no work is named for demotion. The report runs at session start, stays silent unless something is ripe, and speaks at most once a day - a report that speaks every time gets skipped.

The first two blocks it graded bought 0 tool calls each, on the same pattern that had been demoted by hand that morning after measuring 888 closings. The loop reproduced a hand measurement in one second.

5 Negative results

These cost the most and are the most useful.

Asking the model how sure it is does not work. It answered 3/3 every time. Re-rolling the same message at temperature 0.8 gave 5/5 agreement. Only token logprobs gave a number that moved.

The number moves, and still cannot decide. On the doubt lane it separates well at the extremes: 0.27 on a known false positive against 0.97 on a real stop. But over the 30 closings that arrive with doubts only, the model stops 24, and of the 6 it stops with confidence under 0.75, three are finished reports and three are real cuts. A threshold there trades one win for one loss. Confidence is logged and decides nothing.

A shared GPU breaks determinism. Two test suites running at once gave different verdicts on identical input, because the daemon batches concurrent requests. Diagnosed first as a flaky model, then as a prompt problem - adding an example made it worse. The fix is a machine-wide lock plus a fixed seed.

A regression that was not one. The first A/B run showed three regressions. Both sides had been given a different working directory, so one checker only saw the files on one side. An A/B harness has to pin everything the code reads.

6 Tone is load-bearing

The gate's messages were rewritten from accusation to encouragement: what you did stands, permission does not expire, take the step, you are more than able to finish this. The reasoning is not politeness. An agent that reads an accusation argues with it and spends the woken turn defending itself, which is the exact failure the block was meant to prevent.

The same reasoning keeps the diagnostics out of the message. Confidence scores, which patterns fired, whether the lanes disagreed - all of it is useful for tuning and all of it reads as evidence in a prosecution. It goes to the private log instead. The agent gets the ask; the maintainer gets the numbers.

7 Limits

The corpus is one user's sessions, so the patterns carry that user's idiom - the model was given a River Plate Spanish reading list because "quedó cableado" was being read as work in progress. The grading rule counts tool calls, which rewards an agent that acts and cannot see whether the action was any good. And 8 blocks is a threshold picked for the volume this log receives, not derived.

The loop is self-correcting in one direction only: it finds patterns that block too much. Patterns that block too little leave no trace, because a turn that ended is a turn nobody looked at.